Contents

Android Multi-Threading Lab Manual

Project: MultiTreading
Package: `com.example.multitreading`
Purpose: Learn how background work and UI updates interact on Android, and why the main (UI) thread must never be blocked.

Table of Contents

---

1. Introduction

1.1 What This Lab Teaches

Android runs your user interface on a single **main thread** (also called the UI thread). Any long-running work—network calls, file I/O, heavy computation, or even `Thread.sleep()` for demonstration—must run on a **background thread**. When the background work finishes, you must **post results back to the main thread** before touching `View` objects (`TextView`, `ProgressBar`, `Button`, etc.).

This project demonstrates:

Concept	Where it appears
Background work without freezing the UI	`MainActivity` (launcher) and `FirstFragment` (template)
Updating UI from a background thread safely	`Handler` + `Looper` in `MainActivity`; `runOnUiThread()` in `FirstFragment`
Proving the UI stays responsive	"UI Click Counter" buttons while a 10-second task runs
Thread pool / executor lifecycle	`ExecutorService` in `MainActivity`; `shutdown()` in `onDestroy()`
Navigation between screens (template)	`FirstFragment` ⇄ `SecondFragment` via Navigation Component

1.2 Which Code Runs When You Press Run?

The **launcher activity** declared in `AndroidManifest.xml` is `MainActivity`. It inflates **`activity_main.xml`** directly. That is the screen students see on install/run.

The project also contains a **Navigation + Fragment** template (`content_main.xml`, `nav_graph.xml`, `FirstFragment`, `SecondFragment`). Those files illustrate the **same multithreading ideas inside a Fragment** and fragment navigation, but they are **not** connected to the current `MainActivity` until you refactor `MainActivity` to host `content_main` (see Lab Exercise 6).

1.3 Key Android Rules (Memorize These)

1. **Never block the main thread** — no `Thread.sleep()`, network, or disk work on the UI thread.
2. **Never update Views from a background thread** — use `Handler(Looper.getMainLooper())`, `runOnUiThread()`, `view.post()`, or higher-level APIs (Kotlin coroutines, `LiveData`, etc.).
3. **Shut down executors** when the `Activity` is destroyed to avoid leaks.

---

2. Project Structure (Structural View)

```
MultiTreading/
  app/
     build.gradle.kts          # App module: SDK, View Binding, dependencies
     src/
         main/
             AndroidManifest.xml
             java/com/example/multitreading/
                 MainActivity.java      ← LAUNCHER (active demo)
                 FirstFragment.java     ← Fragment demo (template)
                 SecondFragment.java    ← Navigation target (template)
             res/
                 layout/
                     activity_main.xml  ← Used by MainActivity at runtime
                     content_main.xml   ← Nav host (not used by current MainActivity)
                     fragment_first.xml
                     fragment_second.xml
                 navigation/
                     nav_graph.xml
                 menu/
                     menu_main.xml
                 values/                # strings, colors, themes, dimens
                 values-night/
                 values-v23/
                 xml/                   # backup rules
         androidTest/                    # Instrumented tests
         test/                           # Unit tests
  build.gradle.kts              # Root Gradle plugins
  settings.gradle.kts           # Project name, includes :app
  gradle.properties
  gradle/
     libs.versions.toml         # Version catalog (dependencies)
     wrapper/
```

---

3. Application Flow (Top to Bottom)

Execution order from app start to user interaction (**active path**):

1. Android OS reads AndroidManifest.xml
   → Finds LAUNCHER activity: MainActivity

2. MainActivity.onCreate()
   → ActivityMainBinding.inflate()
   → setContentView(activity_main.xml)

→ Register click listeners on buttons

3. User sees activity_main.xml UI
   - Start Background Task
   - ProgressBar + Status TextView
   - UI Click Counter

```
btn_ui_click                        btn_start_thread
(main thread)                       startBackgroundTask()
clickCount++
update tv_counter                   executorService.execute
```

```
Background thread (worker)
Loop i=1..10:
  Thread.sleep(1000)
  mainThreadHandler.post {
    update ProgressBar + tv_status
  }
```

```
Main thread runs posted Runnables
Final post: "Finished!", re-enable
```

Alternate path (if you wire Navigation): `content_main` → `NavHostFragment` → `FirstFragment` / `SecondFragment` using `Thread` + `runOnUiThread()` instead of `ExecutorService` + `Handler`.

---

4. Configuration Files

Configuration is read **before** your Java code runs. Order of influence: Gradle build → merged manifest → theme → layout inflation.

4.1 `settings.gradle.kts`

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
```

```
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "MultiTreading"
include(":app")
```

4.2 Root build.gradle.kts

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
}
```

4.3 app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.multitreading"
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId = "com.example.multitreading"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
```

```
    buildFeatures {
        viewBinding = true
    }
}

dependencies {
    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)
    implementation(libs.navigation.fragment)
    implementation(libs.navigation.ui)
    testImplementation(libs.junit)
    androidTestImplementation(libs.espresso.core)
    androidTestImplementation(libs.ext.junit)
}
```

Teaching points:

- `viewBinding = true` generates `ActivityMainBinding`, `FragmentFirstBinding`, etc.
- `minSdk = 26` — app runs on Android 8.0+.
- Navigation libraries are present for the fragment template.

4.4 `gradle/libs.versions.toml` (excerpt)

Library	Version
Android Gradle Plugin	9.2.1
AppCompat	1.7.1
Material	1.14.0
Navigation	2.9.8
ConstraintLayout	2.2.1

4.5 `gradle.properties`

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
```

4.6 `AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MultiTreading">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.MultiTreading">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

```
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>

</manifest>
```

Execution link: `MAIN` + `LAUNCHER` → `MainActivity` is the entry point.

4.7 app/src/main/res/navigation/nav_graph.xml

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/FirstFragment">

    <fragment
        android:id="@+id/FirstFragment"
        android:name="com.example.multitreading.FirstFragment"
        android:label="@string/first_fragment_label"
        tools:layout="@layout/fragment_first">

        <action
            android:id="@+id/action_FirstFragment_to_SecondFragment"
            app:destination="@id/SecondFragment" />
    </fragment>
    <fragment
        android:id="@+id/SecondFragment"
        android:name="com.example.multitreading.SecondFragment"
        android:label="@string/second_fragment_label"
        tools:layout="@layout/fragment_second">

        <action
            android:id="@+id/action_SecondFragment_to_FirstFragment"
            app:destination="@id/FirstFragment" />
    </fragment>
</navigation>
```

---

5. Resource Files (XML — values/)

Resources are compiled into `R` and referenced from XML (`@string/...`) and Java (`R.string....`).

5.1 res/values/strings.xml

```
<resources>
    <string name="app_name">MultiTreading</string>
    <string name="action_settings">Settings</string>
    <!-- Strings used for fragments for navigation -->
    <string name="first_fragment_label">First Fragment</string>
    <string name="second_fragment_label">Second Fragment</string>
    <string name="next">Next</string>
    <string name="previous">Previous</string>

    <string name="start_background_task">Start Background Task</string>
```

```
    <string name="status_idle">Status: Idle</string>
    <string name="status_running_initial">Status: Running...</string>
    <string name="status_running">Status: Running (%1$d%%)</string>
    <string name="status_finished">Status: Finished!</string>
    <string name="ui_click_counter">UI Click Counter</string>
    <string name="ui_clicks">UI Clicks: %1$d</string>

    <string name="lorem_ipsum">
        Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam in scelerisque sem. Mauris
        volutpat, dolor id interdum ullamcorper, risus dolor egestas lectus, sit amet mattis purus
        dui nec risus. Maecenas non sodales nisi, vel dictum dolor. Class aptent taciti sociosqu ad
        litora torquent per conubia nostra, per inceptos himenaeos. Suspendisse blandit eleifend
        diam, vel rutrum tellus vulputate quis. Aliquam eget libero aliquet, imperdiet nisl a,
        ornare ex. Sed rhoncus est ut libero porta lobortis. Fusce in dictum tellus.\n\n
        Suspendisse interdum ornare ante. Aliquam nec cursus lorem. Morbi id magna felis. Vivamus
        egestas, est a condimentum egestas, turpis nisl iaculis ipsum, in dictum tellus dolor sed
        neque. Morbi tellus erat, dapibus ut sem a, iaculis tincidunt dui. Interdum et malesuada
        fames ac ante ipsum primis in faucibus. Curabitur et eros porttitor, ultricies urna vitae,
        molestie nibh. Phasellus at commodo eros, non aliquet metus. Sed maximus nisl nec dolor
        bibendum, vel congue leo egestas.\n\n
        Sed interdum tortor nibh, in sagittis risus mollis quis. Curabitur mi odio, condimentum sit
        amet auctor at, mollis non turpis. Nullam pretium libero vestibulum, finibus orci vel,
        molestie quam. Fusce blandit tincidunt nulla, quis sollicitudin libero facilisis et. Integer
        interdum nunc ligula, et fermentum metus hendrerit id. Vestibulum lectus felis, dictum at
        lacinia sit amet, tristique id quam. Cras eu consequat dui. Suspendisse sodales nunc ligula,
        in lobortis sem porta sed. Integer id ultrices magna, in luctus elit. Sed a pellentesque
        est.\n\n
        Aenean nunc velit, lacinia sed dolor sed, ultrices viverra nulla. Etiam a venenatis nibh.
        Morbi laoreet, tortor sed facilisis varius, nibh orci rhoncus nulla, id elementum leo dui
        non lorem. Nam mollis ipsum quis auctor varius. Quisque elementum eu libero sed commodo. In
        eros nisl, imperdiet vel imperdiet et, scelerisque a mauris. Pellentesque varius ex nunc,
        quis imperdiet eros placerat ac. Duis finibus orci et est auctor tincidunt. Sed non viverra
        ipsum. Nunc quis augue egestas, cursus lorem at, molestie sem. Morbi a consectetur ipsum, a
        placerat diam. Etiam vulputate dignissim convallis. Integer faucibus mauris sit amet finibus
        convallis.\n\n
        Phasellus in aliquet mi. Pellentesque habitant morbi tristique senectus et netus et
        malesuada fames ac turpis egestas. In volutpat arcu ut felis sagittis, in finibus massa
        gravida. Pellentesque id tellus orci. Integer dictum, lorem sed efficitur ullamcorper,
        libero justo consectetur ipsum, in mollis nisl ex sed nisl. Donec maximus ullamcorper
        sodales. Praesent bibendum rhoncus tellus nec feugiat. In a ornare nulla. Donec rhoncus
        libero vel nunc consequat, quis tincidunt nisl eleifend. Cras bibendum enim a justo luctus
        vestibulum. Fusce dictum libero quis erat maximus, vitae volutpat diam dignissim.
    </string>
</resources>
```

Usage in Java: `getString(R.string.status_running, progress)` — `%1$d` is replaced by `progress`.

5.2 res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

5.3 res/values/themes.xml

```xml
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.MultiTreading" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.MultiTreading" parent="Base.Theme.MultiTreading" />
</resources>
```

5.4 res/values-night/themes.xml

```xml
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.MultiTreading" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your dark theme here. -->
        <!-- <item name="colorPrimary">@color/my_dark_primary</item> -->
    </style>
</resources>
```

5.5 res/values-v23/themes.xml

```xml
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Theme.MultiTreading" parent="Base.Theme.MultiTreading">
        <!-- Transparent system bars for edge-to-edge. -->
        <item name="android:navigationBarColor">@android:color/transparent</item>
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
    </style>
</resources>
```

5.6 res/values/dimens.xml

```xml
<resources>
    <dimen name="fab_margin">16dp</dimen>
</resources>
```

5.7 res/menu/menu_main.xml

```xml
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="com.example.multitreading.MainActivity">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```

Note: Current ***MainActivity*** *does not inflate this menu; it remains from the project template.*

---

6. Layout Files (XML — res/layout/)

Layouts define the UI **before** Java attaches behavior. View Binding maps `android:id` to fields (e.g. `btn_start_thread` → `binding.btnStartThread`).

6.1 `activity_main.xml` — Used at runtime by MainActivity

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    android:gravity="center_horizontal">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Multi-Threading Demo"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="30dp"/>

    <Button
        android:id="@+id/btn_start_thread"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Start Background Task" />

    <ProgressBar
        android:id="@+id/progress_bar"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:max="100" />

    <TextView
        android:id="@+id/tv_status"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="Status: Idle" />

    <View
        android:layout_width="match_parent"
        android:layout_height="2dp"
        android:background="#CCCCCC"
        android:layout_marginVertical="30dp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Test UI Responsiveness"
        android:layout_marginBottom="10dp"/>

    <Button
        android:id="@+id/btn_ui_click"
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:backgroundTint="#4CAF50"
        android:text="Click Me (UI Counter)" />

    <TextView
        android:id="@+id/tv_counter"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="UI Clicks: 0"
        android:textSize="18sp" />

</LinearLayout>
```

View ID	Role in lab
`btn_start_thread`	Starts 10-step background simulation
`progress_bar`	Visual progress 0–100%
`tv_status`	Text feedback from background task
`btn_ui_click` / `tv_counter`	Prove UI thread is not blocked

6.2 `content_main.xml` — Nav host (template; not used by current MainActivity)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/androi
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment_content_main"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

6.3 `fragment_first.xml` — FirstFragment UI

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FirstFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
```

```
android:padding="16dp">

<Button
    android:id="@+id/button_first"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/next"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

<Button
    android:id="@+id/button_start_thread"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="@string/start_background_task"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/button_first" />

<ProgressBar
    android:id="@+id/progress_bar"
    style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:max="100"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/button_start_thread" />

<TextView
    android:id="@+id/textview_status"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="@string/status_idle"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/progress_bar" />

<Button
    android:id="@+id/button_ui_task"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="@string/ui_click_counter"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/textview_status" />

<TextView
    android:id="@+id/textview_first"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
```

```
            android:text="@string/ui_clicks"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_ui_task" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

6.4 `fragment_second.xml` — SecondFragment UI

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".SecondFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/previous"
            app:layout_constraintBottom_toTopOf="@id/textview_second"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/textview_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_second" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

---

7. Java Source Files (Top-to-Bottom Flow)

Read in this order to match **runtime execution**.

7.1 `MainActivity.java` — Entry point (ACTIVE)

```
package com.example.multitreading;

import android.os.Bundle;
import android.os.Handler;
```

```java
import android.os.Looper;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;
import com.example.multitreading.databinding.ActivityMainBinding;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MainActivity extends AppCompatActivity {

    private ActivityMainBinding binding;
    private int clickCount = 0;

    // Executor for background tasks
    private final ExecutorService executorService = Executors.newSingleThreadExecutor();
    // Handler to post updates to the main thread
    private final Handler mainThreadHandler = new Handler(Looper.getMainLooper());

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Setup View Binding
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        // 1. Background Task Button
        binding.btnStartThread.setOnClickListener(v -> startBackgroundTask());

        // 2. UI Counter Button (to prove the UI is not frozen)
        binding.btnUiClick.setOnClickListener(v -> {
            clickCount++;
            binding.tvCounter.setText("UI Clicks: " + clickCount);
        });
    }

    private void startBackgroundTask() {
        // Prepare UI
        binding.btnStartThread.setEnabled(false);
        binding.tvStatus.setText("Status: Running...");
        binding.progressBar.setProgress(0);

        // Execute task in background thread
        executorService.execute(() -> {
            for (int i = 1; i <= 10; i++) {
                int progress = i * 10;

                try {
                    // Simulate heavy work
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                // Update UI on Main Thread
                mainThreadHandler.post(() -> {
                    binding.progressBar.setProgress(progress);
                    binding.tvStatus.setText("Status: Working... (" + progress + "%)");
```

```
                });
            }

            // Task complete
            mainThreadHandler.post(() -> {
                binding.tvStatus.setText("Status: Finished!");
                binding.btnStartThread.setEnabled(true);
            });
        });
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        // Shutdown executor to prevent memory leaks
        executorService.shutdown();
    }
}
```

Line-by-line execution trace (`startBackgroundTask`)

Step	Thread	What happens
1	Main	Disable start button; reset progress and status
2	Main	`executorService.execute(...)` queues work
3	Worker	Loop 1–10: `Thread.sleep(1000)` simulates work
4	Worker	Each iteration: `mainThreadHandler.post(...)` schedules UI update
5	Main	Handler runs `Runnable`: updates `ProgressBar` and `tv_status`
6	Worker	After loop: final `post` sets "Finished!" and re-enables button
7	Main	`onDestroy`: `executorService.shutdown()`

Why `Handler(Looper.getMainLooper())`?
The `Handler` is bound to the main thread's message queue. `post(Runnable)` runs the runnable on that thread—safe for Views.

Why `ExecutorService` instead of `new Thread()`?
A single-thread executor reuses one worker thread and is easy to shut down. `FirstFragment` uses raw `Thread` for comparison.

7.2 `FirstFragment.java` — Fragment-based demo (TEMPLATE)

Used when `content_main` + `nav_graph` host this fragment.

```
package com.example.multitreading;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;
```

```
import com.example.multitreading.databinding.FragmentFirstBinding;

public class FirstFragment extends Fragment {

    private FragmentFirstBinding binding;

    @Override
    public View onCreateView(
            @NonNull LayoutInflater inflater, ViewGroup container,
            Bundle savedInstanceState
    ) {

        binding = FragmentFirstBinding.inflate(inflater, container, false);
        return binding.getRoot();

    }

    private int uiClickCount = 0;

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        binding.buttonFirst.setOnClickListener(v ->
                NavHostFragment.findNavController(FirstFragment.this)
                        .navigate(R.id.action_FirstFragment_to_SecondFragment)
        );

        binding.buttonStartThread.setOnClickListener(v -> startBackgroundTask());

        binding.buttonUiTask.setOnClickListener(v -> {
            uiClickCount++;
            binding.textviewFirst.setText(getString(R.string.ui_clicks, uiClickCount));
        });
    }

    private void startBackgroundTask() {
        binding.buttonStartThread.setEnabled(false);
        binding.textviewStatus.setText(R.string.status_running_initial); // Need to add this or use generic
        binding.progressBar.setProgress(0);

        new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                final int progress = i * 10;
                try {
                    Thread.sleep(1000); // Simulate long task (1 second)
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                // Update UI from background thread using runOnUiThread
                if (getActivity() != null) {
                    getActivity().runOnUiThread(() -> {
                        binding.progressBar.setProgress(progress);
                        binding.textviewStatus.setText(getString(R.string.status_running, progress));
                    });
                }
```

```
            }

            // Task finished, update UI one last time
            if (getActivity() != null) {
                getActivity().runOnUiThread(() -> {
                    binding.textviewStatus.setText(R.string.status_finished);
                    binding.buttonStartThread.setEnabled(true);
                });
            }
        }).start();
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }

}
```

Compare with MainActivity:

MainActivity	FirstFragment
`ExecutorService`	`new Thread(...).start()`
`Handler.post()`	`getActivity().runOnUiThread()`
Hard-coded status strings in places	`strings.xml` format strings
Activity lifecycle	Fragment lifecycle + null-check `getActivity()`

7.3 `SecondFragment.java` — Navigation only

```
package com.example.multitreading;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;

import com.example.multitreading.databinding.FragmentSecondBinding;

public class SecondFragment extends Fragment {

    private FragmentSecondBinding binding;

    @Override
    public View onCreateView(
            @NonNull LayoutInflater inflater, ViewGroup container,
            Bundle savedInstanceState
    ) {

        binding = FragmentSecondBinding.inflate(inflater, container, false);
        return binding.getRoot();
```

```
    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        binding.buttonSecond.setOnClickListener(v ->
                NavHostFragment.findNavController(SecondFragment.this)
                        .navigate(R.id.action_SecondFragment_to_FirstFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }

}
```

---

8. Complete Lifecycle Flow (Summary)

8.1 Activity lifecycle (MainActivity — active)

```
App Launch
    → Application.onCreate (framework)
    → MainActivity.onCreate
        → inflate activity_main
        → set listeners
    → MainActivity.onStart / onResume  (visible, interactive)

User: Start Background Task
    → UI updates on main thread (disable button)
    → Worker thread runs (ExecutorService)
    → Handler posts UI updates → main thread

User: rotates device / leaves app
    → onPause → onStop → (maybe onDestroy if finished)
    → onDestroy: executorService.shutdown()

User: UI Counter clicks
    → Entirely on main thread (onClick)
```

8.2 Fragment lifecycle (when Navigation is wired)

```
NavHostFragment created
    → FirstFragment.onCreateView → onViewCreated
    → User starts thread → new Thread + runOnUiThread
    → Navigate to SecondFragment
        → FirstFragment.onDestroyView (binding = null)
        → SecondFragment.onCreateView → onViewCreated
    → Back navigation reverses the stack
```

8.3 Thread lifecycle (background task)

```
[Main] startBackgroundTask()
[Main] executor.execute(task)
```

```
[Worker] sleep + compute progress
[Worker] handler.post(uiUpdate)   message queue   [Main] run uiUpdate
[Worker] loop ends
[Worker] handler.post(completion)    [Main] enable button, set "Finished!"
```

---

9. File Relationship Diagram

```
flowchart TB
    subgraph build [Build Time]
        Gradle[build.gradle.kts + libs.versions.toml]
        Manifest[AndroidManifest.xml]
        Gradle --> APK[APK]
        Manifest --> APK
    end

    subgraph resources [Resources]
        Strings[strings.xml]
        Themes[themes.xml]
        ActLayout[activity_main.xml]
        Frag1Layout[fragment_first.xml]
        Frag2Layout[fragment_second.xml]
        NavGraph[nav_graph.xml]
        ContentMain[content_main.xml]
    end

    subgraph active [Runtime - Active Path]
        MA[MainActivity.java]
        Binding[ActivityMainBinding]
        Exec[ExecutorService]
        Handler[Handler + Main Looper]
        MA --> Binding
        Binding --> ActLayout
        MA --> Exec
        MA --> Handler
        Exec --> Handler
        Handler --> ActLayout
    end

    subgraph template [Runtime - Template Path]
        CM[content_main.xml]
        NF[NavHostFragment]
        FF[FirstFragment.java]
        SF[SecondFragment.java]
        CM --> NF
        NavGraph --> NF
        NF --> FF
        NF --> SF
        FF --> Frag1Layout
        SF --> Frag2Layout
        Strings --> FF
        Strings --> SF
    end

    Manifest -->|LAUNCHER| MA
    Themes --> MA
```

Legend:

- **Solid active path:** Manifest → `MainActivity` → `activity_main.xml` → `ExecutorService` / `Handler`
- **Dashed template path:** `content_main` → Navigation → Fragments (`Thread` / `runOnUiThread`)

---

10. Lab Exercises for Students

Exercise 1 — Observe non-blocking UI (15 min)

1. Run the app on emulator or device.
2. Tap **Start Background Task**.
3. While progress runs (~10 seconds), tap **Click Me (UI Counter)** repeatedly.
4. **Record:** Does the counter increase during the background task? Why?

Expected: Yes. Background work runs on another thread; the main thread still processes clicks.

Exercise 2 — Intentional ANR (10 min)

1. In `MainActivity.startBackgroundTask()`, temporarily move `Thread.sleep(1000)` **outside** the `executorService.execu` block (run it on the main thread inside `onClick`).
2. Run the app and start the task.
3. **Record:** What happens after ~5 seconds? (ANR dialog)

Restore the correct version after the experiment.

Exercise 3 — Break the UI thread rule (10 min)

1. Comment out `mainThreadHandler.post` and update `binding.progressBar` directly inside the worker loop.
2. Run with **StrictMode** or watch Logcat for warnings/errors.

Discuss: Why does Android forbid cross-thread View access?

Exercise 4 — Compare Handler vs runOnUiThread (20 min)

1. Read `MainActivity` and `FirstFragment` side by side.
2. Write a short table: API used, thread created, lifecycle cleanup, string resources.

Exercise 5 — Use string resources in MainActivity (15 min)

1. Replace hard-coded `"Status: Running..."` and `"Status: Working... (%)"` in `MainActivity` with `R.string.status_running_initial` and `R.string.status_running` (same as `FirstFragment`).
2. Rebuild and verify text still updates correctly.

Exercise 6 — Enable the Fragment navigation path (30–45 min)

1. Change `MainActivity.onCreate` to inflate `content_main` instead of `activity_main` (use `ContentMainBinding` or `setContentView(R.layout.content_main)`).
2. Remove duplicate multithreading UI from the activity (only fragments should demo it).
3. Run app: confirm **First Fragment** appears with background task + Next button.
4. Navigate to Second Fragment and back.

Exercise 7 — Cancel background work (advanced, 30 min)

1. Keep a reference to the running task or use a `volatile boolean cancelled`.
2. Add a **Cancel** button that sets the flag and interrupts sleep.
3. Re-enable UI when cancelled.

Exercise 8 — Written assessment (10 min)

Answer in 3–5 sentences each:

1. What is the main thread responsible for?
2. Name three ways to send work results back to the main thread.
3. Why call `executorService.shutdown()` in `onDestroy()`?

---

11. Prerequisites Checklist

Before starting the lab, ensure you can check every item:

Environment

- ☐ **Android Studio** installed (version compatible with AGP 9.2.x)
- ☐ **JDK 11** or newer configured in Android Studio
- ☐ **Android SDK Platform 36** (or project `compileSdk`) installed via SDK Manager
- ☐ Emulator **API 26+** or physical device with USB debugging

Knowledge

- ☐ Basic Java (classes, loops, anonymous lambdas / listeners)
- ☐ XML layouts (`LinearLayout`, `Button`, `TextView`, `ProgressBar`)
- ☐ Activity lifecycle: `onCreate`, `onDestroy` at minimum
- ☐ Understanding of `R` class and `@string/` references

Project setup

- ☐ Open folder `MultiTreading` in Android Studio
- ☐ Gradle sync completes without errors
- ☐ Run configuration targets `app`
- ☐ App installs as **MultiTreading**

Optional (for fragment exercises)

- ☐ Navigation Component basics (`NavController`, `nav_graph.xml`)
- ☐ Fragment lifecycle (`onCreateView`, `onViewCreated`, `onDestroyView`)

---

12. Troubleshooting

Problem	Likely cause	Solution
Gradle sync failed	Missing SDK or network	Install SDK 36; check proxy; **File → Sync Project with Gradle Files**
`Cannot resolve symbol R`	Build error in XML/Java	Fix red errors in `res/`; **Build → Rebuild Project**

Problem	Likely cause	Solution
App crashes on button press with `CalledFromWrongThreadException`	UI updated from worker thread	Wrap View changes in `Handler.post()` or `runOnUiThread()`
UI freezes / ANR	`Thread.sleep` or heavy work on main thread	Move work to `executorService` or `new Thread()`
Progress bar does not move	Handler not used or wrong binding	Verify `mainThreadHandler.post { ... }` runs
Fragment UI never appears	`MainActivity` still uses `activity_main`	Complete Exercise 6; use `content_main` + NavHost
`NullPointerException` in `FirstFragment`	`getActivity()` is null after detach	Keep `if (getActivity() != null)` checks; cancel task in `onDestroyView`
View Binding errors after rename	IDs changed in XML	Rebuild; binding field names follow camelCase of IDs
Executor leak warning	Activity destroyed while executor runs	Call `shutdown()` in `onDestroy`; consider cancelling tasks
Dark theme looks wrong	`values-night/themes.xml`	Customize `Base.Theme.MultiTreading` for night qualifiers

Logcat tips

- Filter tag: `AndroidRuntime` for crash stack traces.
- Search for `StrictMode` or `Choreographer` skipped frames when diagnosing jank.

Getting help in lab

1. Read the stack trace **bottom to top** (your code lines first).
2. Confirm which layout is inflated (`activity_main` vs `content_main`).
3. Ask: *Which thread is this line running on?*

Appendix A — View Binding ID Map

XML `android:id`	Generated binding field (camelCase)
`btn_start_thread`	`btnStartThread`
`btn_ui_click`	`btnUiClick`
`progress_bar`	`progressBar`
`tv_status`	`tvStatus`
`tv_counter`	`tvCounter`
`button_start_thread`	`buttonStartThread`
`textview_status`	`textviewStatus`

Appendix B — Exporting This Manual

- Open `docs/LAB_MANUAL.md` in Android Studio or any Markdown viewer.
- Export to PDF: **File → Print** (Chrome/Edge print to PDF), or use Pandoc:
  `pandoc docs/LAB_MANUAL.md -o MultiTreading_Lab_Manual.pdf`

Document generated for the MultiTreading Android lab. Package: `com.example.multitreading` *· Min SDK 26 · Target SDK 36.*